

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Избранные главы информатики»

ОТЧЕТ
к лабораторной работе №4
на тему:
**«РАБОТА С ФАЙЛАМИ, КЛАССАМИ, СЕРИАЛИЗАТОРАМИ,
РЕГУЛЯРНЫМИ ВЫРАЖЕНИЯМИ И СТАНДАРТНЫМИ
БИБЛИОТЕКАМИ»**
БГУИР 6-05-0612-02 78

Выполнил студент группы 453503
МОРОЗОВА Эвелина Сергеевна

(дата, подпись студента)

Проверил доцент каф. Информатики
ЖВАКИНА Анна Васильевна

(дата, подпись преподавателя)

Минск 2026

1 ЦЕЛЬ

Освоить базовый синтаксис языка Python, приобрести навыки работы с файлами, классами, сериализаторами, регулярными выражениями и стандартными библиотеками и закрепить их на примере разработки интерактивных приложений.

2 ВЫПОЛНЕНИЕ ЗАДАНИЯ

В качестве среды разработки использовался PyCharm.

Для проверки ввода были написаны следующие функции:

```
"""
Module for input validation and sequence initialization.
Provides functions for safe user input with type checking and range
validation.
"""
from utils.constants import ALLOWED_COLORS, DEFAULT_COLOR

def input_int(min_value: int, max_value: int, message: str) -> int | None:
    while True:
        try:
            value = int(input(message))
            if min_value <= value <= max_value:
                return value
            else:
                print("Invalid value. Please try again.")
                continue

        except ValueError:
            print("Invalid value. Please try again.")
            continue

def input_str(message: str) -> str | None:
    """Input string."""
    while True:
        try:
            value = input(message)
            if value != "":
                return value
            else:
                continue
        except ValueError:
            print("Invalid value. Please try again.")

def input_float(min_value: float, max_value: float, message: str, including:
bool = False) -> float | None:
    """Input float number from min to max."""
```

```

while True:
    try:
        value = float(input(message))
        if including:
            if min_value <= value <= max_value:
                return value
            else:
                continue
        else:
            if min_value < value < max_value:
                return value
            else:
                continue
    except ValueError:
        print("Invalid value. Please try again.")

def check_color():
    color = input_str("Enter color: ").lower()
    if color not in ALLOWED_COLORS:
        print(f"\n!!! Invalid color. Default color is {DEFAULT_COLOR}. !!!")
        color = DEFAULT_COLOR
    return color

```

Для вывода значений:

```

"""
This module implements console output functions.
"""

def console_menu():
    print("\nChoose an option:")
    print("0. Exit")

    print("1. Using serializer put data in file. Available options: reading,
searching, sorting. \n    Formats CSV, module pickle")
    print("2. Text analyzer. Using regular expressions get info and export it
in zipfile")
    print("3. Using task from Lab3 add new parameters ...")
    print("4. Geometric figures")
    print("5. NumPy")
    print("6. Pandas\n")

def task1_info():
    print("\nTask 1 info: Using serializer put data in file. Available
options: reading,\nsearching, sorting. Formats CSV, module pickle\n")

def print_all_candidates_info(candidates: list) -> None:
    print("=" * 35)
    for candidate in candidates:
        print(candidate)
    print("=" * 35, "\n")

```

```

def print_passed_candidates_info(passed_candidates: list, amount_of_votes:
int, pass_quote: float) -> None:
    print("2) Passed candidates: ")
    print(f"For passing candidate needs to get more than {amount_of_votes *
pass_quote} votes")
    if len(passed_candidates) > 0:
        print_all_candidates_info(passed_candidates)
    else:
        print("No candidate to vote. Need to hold repeat elections\n")

def print_sorted_candidates_info(passed_candidates: list) -> None:
    print("3) Sorted (by amount of votes) candidates: ")
    sorted_candidates = sorted(passed_candidates)
    if len(passed_candidates) > 0:
        print_all_candidates_info(sorted_candidates)
    else:
        print("No candidate to vote. Need to hold repeat elections")

def task2_info():
    print("\nTask 2 info: ")

def task3_info():
    print("\nTask 3 info: ")

def print_table(x: float, n: int, f: float, mf: float, eps: float):
    """Print a table for task3."""
    print(f"{x} | {n} | {f} | {mf} | {eps}")

def task4_info(allowed_colors: list) -> None:
    print("\nTask 4 info: \nAvailable colors: ")
    for index, color in enumerate(allowed_colors):
        if index == len(allowed_colors) - 1:
            print(color)
        else:
            print(color, end=", ")
    print()

def task5_info():
    print("\nTask 5 info: ")

def task6_info():
    print("\nTask 6 info: ")

```

Меню программы:

```

"""
Lab work 4. File work, classes, serializers, regular expressions, standard
libraries.
Version 1.0
Developer: Morozova E.S.
Date: 12.04.2026
"""
from task1 import task1_main

```

```

from task2 import task2_main
from task3 import task3_main
from task4 import task4_main
from task5 import task5_main
from task6 import task6_main
from utils.input import input_int
from utils.output import console_menu

def main():
    while True:
        console_menu()
        choice = input_int(0, 6, "Input num of task: ")
        match choice:
            case 0:
                break
            case 1:
                task1_main.run()
            case 2:
                task2_main.run()
            case 3:
                task3_main.run()
            case 4:
                task4_main.run()
            case 5:
                task5_main.run()
            case 6:
                task6_main.run()

if __name__ == "__main__":
    main()

```

1. Исходные данные представляют собой словарь. Необходимо поместить их в файл, используя сериализатор. Организовать считывание данных, поиск, сортировку в соответствии с индивидуальным заданием. Обязательно использовать классы. Реализуйте два варианта: 1) формат файлов CSV; 2) модуль pickle. Реализуйте таблицу выборов (фамилия кандидата и количество набранных голосов). Всего избирателей 2000. Определить, кто из кандидатов прошел, или необходимо проводить повторные выборы (должно быть набрано не менее $\frac{1}{3}$ голосов от общего количества). Выведите информацию о кандидате, введенном с клавиатуры.

```

"""
Task 1. Module provides class Candidate.
"""
class Candidate:
    """Candidate with name and votes. Supports comparison by votes."""
    def __init__(self, name: str, number_of_votes: int) -> None:
        """Initialize candidate."""
        self.__name = name

```

```

        self.__number_of_votes = number_of_votes

    def __str__(self) -> str:
        """Return 'Candidate: {name}, {votes} votes'."""
        return f"Candidate: {self.__name}, {self.__number_of_votes} votes"

    def __gt__(self, other) -> bool:
        """Compare by votes (greater than)."""
        return self.__number_of_votes > other.get_number_of_votes()

    def __lt__(self, other) -> bool:
        """Compare by votes (less than)."""
        return self.__number_of_votes < other.get_number_of_votes()

    def get_name(self) -> str:
        """Return candidate name."""
        return self.__name

    def get_number_of_votes(self) -> int:
        """Return vote count."""
        return self.__number_of_votes

    def set_number_of_votes(self, number_of_votes: int) -> None:
        """Set vote count."""
        if 0 <= number_of_votes <= 2000:
            self.__number_of_votes = number_of_votes

"""
Task 1. Module for CSV parsing.
"""
import csv

from task1.candidate import Candidate
from task1.parser_base import ParserBase

class CsvParser(ParserBase):
    """CSV serializer/deserializer for Candidate objects."""
    def __init__(self, filename: str):
        """Set path to CSV file."""
        self.file_name = filename

    def serialize(self, candidates: list):
        """Save list of Candidate objects to CSV file."""
        candidates_dicts_list = []
        for candidate in candidates:
            candidates_dicts_list.append({"name": candidate.get_name(),
"number_of_votes": candidate.get_number_of_votes()})

        with open(self.file_name, 'w', newline='') as csvfile:
            columns = ['name', 'number_of_votes']
            writer = csv.DictWriter(csvfile, fieldnames=columns)
            writer.writeheader()
            writer.writerows(candidates_dicts_list)

```

```

def deserialize(self):
    """Load list of Candidate objects from CSV file."""
    with open(self.file_name, 'r', newline='') as csvfile:
        reader = csv.DictReader(csvfile)
        candidates = []
        for row in reader:
            candidates.append(Candidate(row['name'],
int(row['number_of_votes'])))
        return candidates

"""
Task 1. Module for ElectionsService.
"""

from task1.candidate import Candidate
from task1.csv_parser import CsvParser
from task1.pickle_parser import PickleParser
from task1.seed_service import SeedService
from        utils.output        import        print_all_candidates_info,
print_passed_candidates_info, print_sorted_candidates_info

class ElectionsService:
    """
    Main service for election management.
    Handles voting, candidate serialization, and candidate search.
    """

    AMOUNT_OF_VOTERS = 2000
    PASS_QUOTE = 0.333

    def __init__(self):
        """Initialize empty candidates list and load initial data."""
        self.__candidates = []
        self._initialize_candidates()

    def _initialize_candidates(self):
        """Load hardcoded candidate data from SeedService and create
Candidate objects."""
        seed_service = SeedService()
        candidates_dict = seed_service.initialize()
        self.__candidates = [Candidate(name, votes) for name, votes in
candidates_dict.items()]

    def _get_candidates(self) -> list:
        """Return list of all candidates."""
        return self.__candidates

    def voting(self) -> None:
        """
        Main voting process:
        1. Serialize candidates to CSV and pickle
        2. Deserialize from pickle
        3. Display all candidates

```

```

4. Determine who passed (votes > 2000 * 0.333)
5. Display passed candidates
"""
csv_parser = CsvParser("task1/candidates.csv")
pickle_parser = PickleParser("task1/candidates.pickle")

candidate_object_list = self._get_candidates()

csv_parser.serialize(candidate_object_list)
pickle_parser.serialize(candidate_object_list)

candidates_from_pickle = pickle_parser.deserialize()
print("1) All candidates:")
print_all_candidates_info(candidates_from_pickle)

passed_candidates = []

for c in candidates_from_pickle:
    if c.get_number_of_votes() > ElectionsService.AMOUNT_OF_VOTERS *
ElectionsService.PASS_QUOTE:
        passed_candidates.append(c)

        print_passed_candidates_info(passed_candidates,
ElectionsService.AMOUNT_OF_VOTERS, ElectionsService.PASS_QUOTE)

def sorted_candidates(self):
    """Display candidates sorted by number of votes."""
    print_sorted_candidates_info(self._get_candidates())

def search_candidate(self, name: str):
    """
    Search for a candidate by name (case-insensitive).

    Args:
        name: Candidate name to search for

    Prints candidate info if found, otherwise "No such candidate".
    """
    for candidate in self.__candidates:
        if candidate.get_name().lower() == name.lower():
            print(candidate)
            break
    else:
        print("No such candidate")

"""
Task 1. Module for ParserBase.
"""
import abc

class ParserBase(abc.ABC):
    """Abstract base class for serialization/deserialization."""
    @abc.abstractmethod

```



```

    def serialize(self, candidates: list):
        """Save candidates list to storage."""
        pass
    @abc.abstractmethod
    def deserialize(self):
        """Load candidates list from storage and return it."""
        pass

"""
Task 1. Module for PickleParser.
"""
import pickle

from task1.parser_base import ParserBase

class PickleParser(ParserBase):

    def __init__(self, file_name):
        """Initialize PickleParser."""
        self.file_name = file_name

    def serialize(self, candidates: list):
        """Serialize PickleParser."""
        with open(self.file_name, "wb") as f:
            pickle.dump(candidates, f)

    def deserialize(self):
        """Deserialize PickleParser."""
        candidates = []
        with open(self.file_name, "rb") as f:
            candidates = pickle.load(f)

        return candidates

"""
Task 1. Module for SeedService.
"""
class SeedService:
    def initialize(self) -> dict:
        """Initialize SeedService."""
        candidates = {"Ivanov": 850, "Petrov": 620, "Sidorov": 430, "Kozlov":
720, "Smirnov": 380}
        return candidates

"""
Lab work 4. File work, classes, serializers, regular expressions, standard
libraries.
Task 1. Election vote analysis.
Version 1.0
Developer: Morozova E.S.
Date: 12.04.2026
"""

```

```

import csv
import pickle

from task1.elections_service import ElectionsService
from utils.input import input_str
from utils.output import task1_info

def run() -> None:
    """Execute task1. """
    task1_info()
    try:
        elections = ElectionsService()
        elections.voting()
        elections.sorted_candidates()
        print("4) Searching for candidate")
        name = input_str("Enter name: ")
        elections.search_candidate(name)
    except csv.Error as err:
        print(f"Parsing csv error: {err}")
    except FileNotFoundError as err:
        print(f"File not found: {err}")
    except PermissionError as err:
        print(f"Permission error: {err}")
    except pickle.PicklingError as err:
        print(f"Serialization error: {err}")
    except pickle.UnpicklingError as err:
        print(f"Deserialization error: {err}")
    except Exception as e:
        print(f"Exception: {e}")

```

```
Task 1 info: Using serializer put data in file. Available options: reading,
searching, sorting. Formats CSV, module pickle
```

```
1) All candidates:
```

```
=====
Candidate: Ivanov, 850 votes
Candidate: Petrov, 620 votes
Candidate: Sidorov, 430 votes
Candidate: Kozlov, 720 votes
Candidate: Smirnov, 380 votes
=====
```

```
2) Passed candidates:
```

```
For passing candidate needs to get more than 666.0 votes
```

```
=====
Candidate: Ivanov, 850 votes
Candidate: Kozlov, 720 votes
=====
```

```
3) Sorted (by amount of votes) candidates:
```

```
=====
Candidate: Smirnov, 380 votes
Candidate: Sidorov, 430 votes
Candidate: Petrov, 620 votes
Candidate: Kozlov, 720 votes
Candidate: Ivanov, 850 votes
=====
```

```
4) Searching for candidate
```

```
Enter name: kozlov
```

```
Candidate: Kozlov, 720 votes
```

2. В соответствии с заданием своего варианта составить программу для анализа текста. Считать из исходного файла текст. Используя регулярные выражения получить искомую информацию (см. условие), вывести ее на экран и сохранить в другой файл. Заархивировать файл с результатом с помощью модуля zipfile и обеспечить получение информации о файле в архиве. Также выполнить общее задание – определить и сохранить в файл с результатами:

- количество предложений в тексте;
- количество предложений в тексте каждого вида отдельно (повествовательные, вопросительные и побудительные);
- среднюю длину предложения в символах (считаются только слова);
- среднюю длину слова в тексте в символах;
- количество смайликов в заданном тексте. Смайликом будем считать последовательность символов, удовлетворяющую условиям: первым

символом является либо «;» (точка с запятой) либо «:» (двоеточие) ровно один раз; далее может идти символ «-» (минус) сколько угодно раз (в том числе символ минус может идти ноль раз); в конце обязательно идет некоторое количество (не меньше одной) одинаковых скобок из следующего набора: «(», «)», «[», «]»; внутри смайлика не может встречаться никаких других символов. Например, эта последовательность является смайликом: «;------[[[[[[[». Эти последовательности смайликами не являются: «]», «;-»,«:»,«)». Вывести все телефоны длиной 9 знаков и начинающиеся на 29 Из заданной строки получить список слов, у которых вторая буква согласная, а третья – гласная определить число слов, ограниченных пробелами; определить, сколько раз повторяется каждая буква; вывести по алфавиту словосочетания, отделенные запятыми.

```
"""
Task 2. Module for file service operations.
"""

import re
from collections import Counter
from zipfile import ZipFile

class FileService:
    """Main service for text analysis and reporting."""
    def __init__(self):
        """Initialize file paths, read text, count sentences."""
        self.__input_filename = "task2/input.txt"
        self.__output_filename = "task2/output.txt"
        self.__output_zip_filename = "task2/output-zip-file.zip"
        self.__text = self._read_file()
        self.__sentences_counter = self.count_sentences()

    def __str__(self):
        """Return formatted report as string."""
        return str("General tasks:\n"
                    f"1) Narrative(.) sentences: {self.count_narrative_sentences()}\n"
                    f"2) Interrogative(?) sentences: {self.count_interrogative_sentences()}\n"
                    f"3) Exclamative(!) sentences: {self.count_exclamative_sentences()}\n"
                    f"4) Total sentences count: {self.count_sentences()}\n"
                    f"5) Average sentences length(only symbols in words counted): {self.calculate_average_sentences_length():.4f}\n"
                    f"6) Average words length: {self.calculate_average_words_length():.4f}\n"
                    f"7) Smileys count: {self.count_smileys()}\n\n"
                    "Individual tasks:\n")
```

```

        f"1) Phone numbers 9 digits, started with 29 (29*****):
{self.find_phone_numbers()}\n"
        f"2) Amount of words where second letter is consonant,
third - vowel: {self.get_specified_words()}\n"
        f"3) Amount of words that are bounded with spaces:
{self.count_words_bounded_by_spaces()}\n"
        f"4) Determine how many times each letter appears:
\n\t{self.format_letters_occurrences()}\n"
        f"5) Get all phrases separated by commas in alphabet
order: \n\t{", ".join(self.get_alphabetizes_phrases_separated_by_commas())}"

    def _read_file(self):
        """Read input file and return content."""
        with open(self.__input_filename, 'r') as file:
            return file.read()

    def save_info_to_file(self):
        """Save report to output file."""
        with open(self.__output_filename, 'w') as file:
            file.write(self.__str__())

    def archive_file(self):
        """Archive output file into ZIP."""
        with ZipFile(self.__output_zip_filename, 'w') as zip_file:
            zip_file.write(self.__output_filename, arcname="results.txt")

    def show_archived_info(self):
        """Display ZIP archive contents info."""
        with ZipFile(self.__output_zip_filename, 'r') as zip_file:
            for info in zip_file.infolist():
                print(f"File name: {info.filename}, Date: {info.date_time},
Size: {info.file_size} bytes")

    def count_sentences(self):
        """Total sentences = narrative + interrogative + exclamative."""
        return self.count_narrative_sentences() +
self.count_interrogative_sentences() + self.count_exclamative_sentences()

    def count_narrative_sentences(self):
        """Count sentences ending with dot (not part of number)."""
        counter = len(re.findall(r'\.(?![0-9])', self.__text))
        return counter

    def count_interrogative_sentences(self):
        """Count sentences ending with question mark."""
        counter = len(re.findall(r'\?', self.__text))
        return counter

    def count_exclamative_sentences(self):
        """Count sentences ending with exclamation mark."""
        counter = len(re.findall(r'!', self.__text))
        return counter

```

```

def calculate_average_sentences_length(self):
    """Average letters per sentence."""
    letters = len(re.findall(r"[a-zA-Z]", self.__text))
    return letters / self.__sentences_counter

def calculate_average_words_length(self):
    """Average letters per word."""
    letters = len(re.findall(r"[a-zA-Z]", self.__text))
    words = len(re.findall(r"\b[^\d-9_][a-zA-Z']*\b", self.__text))
    return letters / words

def count_smileys(self):
    """Count smileys: ; or :, optional -, repeated same bracket at
end."""
    counter = len(re.findall(r"[;:]-*([\[\]\(\)])\1*(?=[,.\s]|$)",
self.__text))
    return counter

def find_phone_numbers(self):
    """Count 9-digit numbers starting with 29."""
    counter = len(re.findall(r"\b29\d{7}\b", self.__text))
    return counter

def get_specified_words(self):
    """Count words where 2nd letter consonant, 3rd vowel."""
    consonants = "QWRTPSDFGHJKLZXCVBNMqwrtpsdfghjklzxcvbnm"
    vowels = "EYUIOAeyuioa"
    counter = len(re.findall(rf"\b[a-zA-Z][{consonants}][{vowels}]\w*\b",
self.__text))
    return counter

def count_words_bounded_by_spaces(self):
    """Count words separated by spaces (word boundaries)."""
    counter = len(re.findall(r"\b[\w']+\b", self.__text))
    return counter

def calculate_letters_occurrences(self):
    """Return Counter of letter frequencies."""
    letters_appearances = Counter(re.findall(r"[a-zA-Z]", self.__text))
    return letters_appearances

def get_alphabetizes_phrases_separated_by_commas(self):
    """Extract phrases after 'Phrases:', split by comma, sort
alphabetically."""
    line = re.findall(r"(?<=Phrases: ).+(?=.)", self.__text)
    phrases = re.split(r",", line[0])
    return sorted(phrases)

def format_letters_occurrences(self):
    """Format letter frequencies as readable string."""
    result = ""
    for key, value in
sorted(self.calculate_letters_occurrences().items()):

```

```

        result += f"{key}: {value}, "
    return result

"""
Lab work 4. File work, classes, serializers, regular expressions, standard
libraries.
Task 2. Text analysis using regular expressions.
Version 1.0
Developer: Morozova E.S.
Date: 12.04.2026
"""

from task2.file_operations_service import FileService

def run():
    """Execute task 2: analyze text, save results, create archive."""
    try:
        file_service = FileService()
        print("\nFILE SERVICE INFO:\n")
        print(file_service)
        print("\nSaving results to file...")
        file_service.save_info_to_file()
        print("\nResult archivation...")
        file_service.archive_file()
        print("\nArchived file info:")
        file_service.show_archived_info()
    except FileNotFoundError as err:
        print(f"File not found: {err}")
    except PermissionError as err:
        print(f"Permission error: {err}")
    except Exception as e:
        print(f"Exception: {e}")

```

```

FILE SERVICE INFO:
General tasks:
1) Narrative(.) sentences: 9
2) Interrogative(?) sentences: 3
3) Exclamative(!) sentences: 4
4) Total sentences count: 16
5) Average sentences length(only symbols in words counted): 18.1250
6) Average words length: 4.0278
7) Smileys count: 10

Individual tasks:
1) Phone numbers 9 digits, started with 20 (20*****): 3
2) Amount of words where second letter is consonant, third - vowel: 27
3) Amount of words that are bounded with spaces: 76
4) Determine how many times each letter appears:
   A: 1, D: 1, F: 1, G: 1, H: 4, I: 1, L: 1, M: 1, P: 1, S: 1, T: 1, W: 2, a: 26, b: 5, c: 3, d: 11, e: 42, f: 5, g: 6, h: 11, i: 12, k: 4, l: 20, m: 5, n: 17, o: 19, p: 2, r: 24, s: 19, t: 20, u: 6, v: 1, w: 7, y: 8, z: 1,
5) Get all phrases separated by commas in alphabet order:
   blue sky, bright sun, fresh air, green grass, tall tree, warm breeze

Saving results to file...
Result archivation...
Archived file info:
File name: results.txt, Date: (2026, 4, 15, 13, 25, 20), Size: 859 bytes

```

3. В соответствии с заданием своего варианта доработать программу из ЛР3, используя класс и обеспечить:

- а) определение дополнительных параметров среднее арифметическое элементов последовательности, медиана, мода, дисперсия, СКО последовательности;

б) с помощью библиотеки matplotlib нарисовать графики разных цветов в одной координатной оси:

- график по полученным данным разложения функции в ряд, представленным в таблице,
- график соответствующей функции, представленной с помощью модуля math.

Обеспечить отображение координатных осей, легенды, текста и аннотации.

x	n	$F(x)$	$Math F(x)$	eps

Здесь x – значение аргумента, $F(x)$ – значение функции, n – количество просуммированных членов ряда, $Math F(x)$ – значение функции, вычисленное с помощью модуля math.

в) сохранить графики в файл

$$\arcsin x = \sum_{n=0}^{\infty} \frac{(2n)!}{4^n (n!)^2 (2n+1)} x^{2n+1} = x + \frac{x^3}{6} + \frac{3x^5}{40} + \dots, |x| < 1$$

```

"""
Task 3. Series Calculation Service.
"""

import statistics
from math import asin, fabs
from utils.output import print_table

class SeriesCalculationService:
    """Service for calculating arcsin series and its statistics."""
    def __init__(self, x: float, eps: float):
        """
        Initialize series calculator.
        Args:
            :param x: Value in range (-1, 1) for arcsin calculation
            :param eps: Precision threshold for series convergence
        """
        self.__x = x
        self.__eps = eps
        self.__series_values = self.calculate_series()

    def __str__(self) -> str:
        """Return formatted statistics report."""
        return str(f"1) Mean: {self.calculate_mean()}\n"
                  f"2) Mode: {self.calculate_mode()}\n"
                  f"3) Median: {self.calculate_median()}\n"
                  f"4) Variance: {self.calculate_variance()}\n")

```



```

f"5) Standard deviation:

{self.calculate_standard_deviation()})

    def get_series_values(self) -> tuple[float, list[int], list[float],
list[float], int, float]:
        """
        Return complete series calculation result.

        Returns:
            tuple: (x, iterations, series_values, arcsin_values,
break_iteration, break_value)
        """
        return self.__series_values

@property
def series_list(self) -> list[float]:
    """Return list of partial series sums."""
    return self.__series_values[2]

    def calculate_series(self) -> tuple[float, list[int], list[float],
list[float], int, float]:
        """
        Compute arcsin series expansion.

        Returns:
            tuple: (x, iterations, series_values, arcsin_values,
break_iteration, break_value)
        """
        series = self.__x
        a = self.__x

        break_iteration = -1
        break_value = -1
        iterations = []
        series_values = []
        arcsin_values = []
        arcsin_value = asin(self.__x)
        for i in range(500):
            print_table(self.__x, i, series, arcsin_value, self.__eps)
            a *= (2*i + 1) ** 2 * self.__x ** 2 / ((2*i + 2)*(2*i + 3))
            series += a

            iterations.append(i)
            series_values.append(series)
            arcsin_values.append(arcsin_value)

            if fabs(arcsin_value - series) < self.__eps:
                break_iteration = i
                break_value = series
                break

        return self.__x, iterations, series_values, arcsin_values,
break_iteration, break_value

```

```

def calculate_mean(self) -> float:
    """Return mean of series values."""
    return statistics.mean(self.series_list)

def calculate_mode(self) -> float:
    """Return mode of series values."""
    return statistics.mode(self.series_list)

def calculate_median(self) -> float:
    """Return median of series values."""
    return statistics.median(self.series_list)

def calculate_variance(self) -> float:
    """Return variance of series values."""
    return statistics.variance(self.series_list)

def calculate_standard_deviation(self) -> float:
    """Return standard deviation of series values."""
    return statistics.stdev(self.series_list)

"""
Lab work 4. File work, classes, serializers, regular expressions, standard
libraries.
Task 3. Series calculation with graphics.
Version 1.0
Developer: Morozova E.S.
Date: 12.04.2026
"""

from statistics import StatisticsError

from task3.series_calculation_service import SeriesCalculationService
from task3.visualisation_service import VisualisationService
from utils.input import input_float
from utils.output import task3_info

def run():
    """Execute task3."""
    task3_info()
    x = input_float(-1.0, 1.0, "Enter x: ")
    eps = input_float(0.0, 1.0, "Enter eps: ")
    calculation_service = SeriesCalculationService(x, eps)
    visualisation_service = VisualisationService()

    try:
        print(calculation_service)
        visualisation_service.draw(calculation_service.get_series_values())
    except StatisticsError as e:
        print(f"Statistics exception: {e}")
    except FileNotFoundError as e:
        print(f"File not found: {e}")
    except PermissionError as e:
        print(f"Permission error: {e}")

```

```

except Exception as e:
    print(f"Exception: {e}")

import matplotlib
import matplotlib.pyplot as plt
import numpy as np

class VisualisationService:
    @staticmethod
    def _draw_plot(n, y_series, y_math, x_value):
        """
        Create base plot with axes, grid, labels and legend.
        Args:
            n: Array of iteration numbers (x-axis)
            y_series: Array of series partial sums
            y_math: Array of math.asin() reference values
            x_value: Original x parameter for title

        Returns:
            tuple: (fig, ax) matplotlib figure and axes objects
        """
        fig, ax = plt.subplots(figsize=(12, 8))
        ax.plot(n, y_series, 'blue', label="Series")
        ax.plot(n, y_math, 'red', label="Math")

        ax.grid(True)
        ax.set_xlabel("n", fontsize=20)
        ax.set_ylabel("y", fontsize=20)
        ax.set_title(f"Convergence of the series to arcsin({x_value})",
            fontsize=20)
        ax.text(0.6, 0.4, "Series convergence graph", transform=ax.transAxes,
            fontsize=12, ha='left', va='top')
        ax.legend(fontsize=16)

        plt.xticks(fontsize=14)
        plt.yticks(fontsize=14)
        return fig, ax

    @staticmethod
    def _add_annotation(ax, break_iteration, break_value):
        """
        Add annotation to axes object.
        Args:
            ax: Matplotlib axes object
            break_iteration: Iteration number where precision was reached
            break_value: Series value at that iteration
        """
        x_text = float(break_iteration - break_iteration*0.01)
        y_text = float(break_value - break_value*0.01)
        ax.annotate('Convergence', xy=(break_iteration, break_value),
            xytext=(x_text, y_text),
            arrowprops=dict(arrowstyle='->'), fontsize=16)

```

```

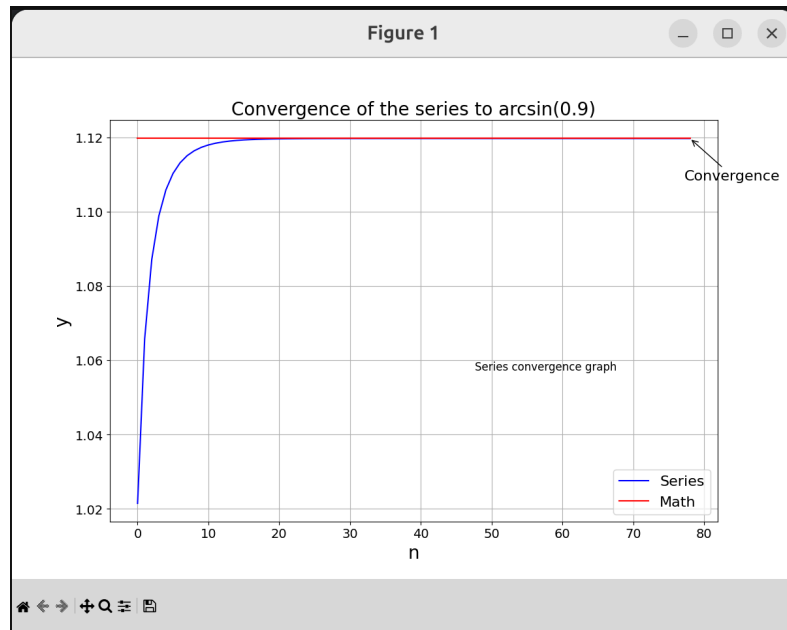
    def draw(self, series_result: tuple[float, list[int], list[float],
list[float], int, float]):
    """
    Main method to plot and save the convergence graph.

    Args:
        series_result: Tuple containing (x, iterations, series_values,
            arcsin_values, break_iteration, break_value)
    """
    x_value = series_result[0]
    n = np.array(series_result[1], float)
    y_series = np.array(series_result[2], float)
    y_math = np.array(series_result[3], float)
    break_iteration = series_result[4]
    break_value = series_result[5]

    fig, ax = self._draw_plot(n, y_series, y_math, x_value)
    self._add_annotation(ax, break_iteration, break_value)

    plt.savefig("task3/convergence.png")
    plt.show()

```



```

0.9 | 1 | 1.0215 | 1.1197695149986342 | 1e-10
0.9 | 2 | 1.06578675 | 1.1197695149986342 | 1e-10
0.9 | 3 | 1.0871392901785715 | 1.1197695149986342 | 1e-10
0.9 | 4 | 1.098909877952009 | 1.1197695149986342 | 1e-10
0.9 | 5 | 1.1059304985321474 | 1.1197695149986342 | 1e-10
0.9 | 6 | 1.1103413384235536 | 1.1197695149986342 | 1e-10
0.9 | 7 | 1.1132165759127661 | 1.1197695149986342 | 1e-10
0.9 | 8 | 1.115143090737799 | 1.1197695149986342 | 1e-10
0.9 | 9 | 1.1164617394377756 | 1.1197695149986342 | 1e-10
0.9 | 10 | 1.1173798015005378 | 1.1197695149986342 | 1e-10
0.9 | 11 | 1.1180279061436984 | 1.1197695149986342 | 1e-10
0.9 | 12 | 1.1184907500746115 | 1.1197695149986342 | 1e-10
0.9 | 13 | 1.1188245317555585 | 1.1197695149986342 | 1e-10
0.9 | 14 | 1.1190672592737634 | 1.1197695149986342 | 1e-10
0.9 | 15 | 1.1192450532658884 | 1.1197695149986342 | 1e-10
0.9 | 16 | 1.1193761106725268 | 1.1197695149986342 | 1e-10
0.9 | 17 | 1.1194732572505282 | 1.1197695149986342 | 1e-10
0.9 | 18 | 1.1195456248871811 | 1.1197695149986342 | 1e-10
0.9 | 19 | 1.1195997731655942 | 1.1197695149986342 | 1e-10
0.9 | 20 | 1.1196404507390623 | 1.1197695149986342 | 1e-10
0.9 | 21 | 1.1196711190617699 | 1.1197695149986342 | 1e-10
0.9 | 22 | 1.1196943168598688 | 1.1197695149986342 | 1e-10
0.9 | 23 | 1.1197119163919371 | 1.1197695149986342 | 1e-10
0.9 | 24 | 1.119725305280855 | 1.1197695149986342 | 1e-10
0.9 | 25 | 1.1197355165926417 | 1.1197695149986342 | 1e-10
0.9 | 26 | 1.1197433225773243 | 1.1197695149986342 | 1e-10
0.9 | 27 | 1.1197497001560060 | 1.1197695149986342 | 1e-10
0.9 | 28 | 1.1197549566986760 | 1.1197695149986342 | 1e-10
0.9 | 29 | 1.1197592500750000 | 1.1197695149986342 | 1e-10
0.9 | 30 | 1.1197627500000000 | 1.1197695149986342 | 1e-10
0.9 | 31 | 1.1197655000000000 | 1.1197695149986342 | 1e-10
0.9 | 32 | 1.1197677500000000 | 1.1197695149986342 | 1e-10
0.9 | 33 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 34 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 35 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 36 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 37 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 38 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 39 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 40 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 41 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 42 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 43 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 44 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 45 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 46 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 47 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 48 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 49 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 50 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 51 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 52 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 53 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 54 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 55 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 56 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 57 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 58 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 59 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 60 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 61 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 62 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 63 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 64 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 65 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 66 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 67 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 68 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 69 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 70 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 71 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 72 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 73 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 74 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 75 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 76 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 77 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 78 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 79 | 1.1197695000000000 | 1.1197695149986342 | 1e-10
0.9 | 80 | 1.1197695000000000 | 1.1197695149986342 | 1e-10

```

4. В соответствии с заданием своего варианта разработать базовые классы и классы наследники.

Требования по использованию классов:

Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для

вычисления

площади

фигуры

(<https://docs.python.org/3/library/abc.html>)

Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры

(<https://docs.python.org/3/library/functions.html#property>)

Класс «Прямоугольник» (Круг, Ромб, Квадрат, Треугольник и т.д.) наследуется от класса

«Геометрическая фигура». Класс должен содержать конструктор по параметрам

«ширина», «высота» (для другого типа фигуры соответствующие параметры, например,

для круга задаем «радиус») и «цвет». В конструкторе создается объект класса «Цвет

фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий

площадь фигуры <https://docs.python.org/3/library/math.html> .

Для класса «Прямоугольник»(тип фигуры в инд. задании)

определить метод, который возвращает в виде строки основные параметры фигуры, ее

цвет и площадь. Использовать метод format (<https://pyformat.info/>)

название фигуры должно задаваться в виде поля данных класса и возвращаться методом

класса.

В корневом каталоге проекта создайте файл main.py для тестирования классов.

Используйте конструкцию, описанную в https://docs.python.org/3/library/__main__.html

Пример объекта: Прямоугольник синего цвета шириной 5 и высотой

Программа должна содержать следующие базовые функции:

- 1) ввод значений параметров пользователем;
- 2) проверка корректности вводимых данных;
- 3) построение, закрашивание фигуры в выбранный цвет, введенный с клавиатуры, и подпись фигуры текстом, введенным с клавиатуры;
- 4) вывод фигуры на экран и в файл

Построить параллелограмм по диагоналям d1, d2 и углу между ними X (в градусах).

```

"""
Task 4. Draw parallelogram.
"""

import math
import matplotlib.pyplot as plt
import numpy as np

class DrawableMixin:
    """Mixin providing drawing functionality for parallelogram."""
    def draw(self, caption: str):
        """
        Draw parallelogram using matplotlib.

        Builds vertices from diagonals and angle, then plots and fills the
        shape.

        Args:
            caption: Text label displayed at the center of the parallelogram.
        """
        angle = self.angle
        half_d1 = self.diagonal1 / 2
        half_d2 = self.diagonal2 / 2

        x_array = [-half_d1, half_d2 * math.cos(angle), half_d1, -half_d2 *
math.cos(angle), -half_d1]
        y_array = [0, half_d2 * math.sin(angle), 0, -half_d2 *
math.sin(angle), 0]
        x = np.array(x_array)
        y = np.array(y_array)

        fig, ax = plt.subplots(figsize=(12, 8))
        ax.grid(True, zorder=0)

        textcolor = "black"
        if self.color == "black" or self.color == "blue":
            textcolor = "white"
        ax.text(0, 0, caption, color=textcolor, fontsize=16, ha="center",
va="center")

        ax.plot(x, y, self.color, zorder=2, label="Parallelogram")
        ax.fill(x, y, self.color, zorder=2)
        ax.axis('equal')

        ax.set_xlabel("x", fontsize=20)
        ax.set_ylabel("y", fontsize=20)
        ax.legend(fontsize=16)

        plt.savefig("task4/parallelogram.png")
        plt.show()

"""
Task 4. Module for figure color.

```

```

"""
class FigureColor:
    """Class to store color of a geometric figure."""
    def __init__(self, color):
        """
        Initialize figure color.

        Args:
            color: Name of the color (e.g., 'red', 'blue', 'green')
        """
        self.__color = color

    @property
    def color(self):
        """
        Get color of the figure.
        Returns:
            str: Color name
        """
        return self.__color

    @color.setter
    def color(self, value):
        """
        Set color of the figure.
        Args:
            value: New color name
        """
        self.__color = value

```

```

"""
Task 4. Module for abstract geometry figure.
"""
import abc

class GeometryFigure(abc.ABC):
    """
    Abstract base class for geometric figures.
    All specific figures must inherit from this class.
    """
    def __init__(self):
        """Initialize geometry figure."""
        pass

    @abc.abstractmethod
    def calculate_area(self):
        """
        Calculate and return area of the figure.

        Returns:
            float: Area of the figure
        """
        pass

```



```

"""
Task 4. Module for parallelogram.
"""

import math
from task4.drawable_mixin import DrawableMixin
from task4.figure_color import FigureColor
from task4.geometry_figure import GeometryFigure

class Parallelogram(GeometryFigure, DrawableMixin):
    """Parallelogram defined by two diagonals and angle between them."""

    name = "Parallelogram"
    def __init__(self, diagonal1, diagonal2, angle, color):
        """
        Initialize parallelogram.

        Args:
            diagonal1: Length of first diagonal
            diagonal2: Length of second diagonal
            angle: Angle between diagonals in degrees
            color: Color name for the figure
        """
        super().__init__()
        self.__diagonal1 = diagonal1
        self.__diagonal2 = diagonal2
        self.__angle = angle
        self.__color = FigureColor(color)

    @classmethod
    def get_name(cls):
        """Return figure name (class method)."""
        return f"\nFigure name: {cls.name}"

    @property
    def diagonal1(self):
        """Get first diagonal length."""
        return self.__diagonal1

    @property
    def diagonal2(self):
        """Get second diagonal length."""
        return self.__diagonal2

    @property
    def angle(self):
        """Get angle in radians (converted from degrees)."""
        return math.radians(self.__angle)

    @property
    def color(self):
        """Get figure color."""
        return self.__color.color

```

```

def calculate_area(self):
    """
    Calculate area using formula:  $0.5 * d1 * d2 * \sin(\text{angle})$ .

    Returns:
        float: Area of the parallelogram
    """
    return 0.5 * self.__diagonal1 * self.__diagonal2 *
math.sin(self.angle)

def get_info(self):
    """
    Return formatted string with figure parameters, color and area.

    Returns:
        str: Formatted info string
    """
    return "Diagonal 1: {}\nDiagonal 2: {}\nAngle: {}° \nArea:
{:.4f}\nColor: {}".format(self.__diagonal1, self.__diagonal2, self.__angle,
self.calculate_area(), self.__color.color)

"""
Lab work 4. File work, classes, serializers, regular expressions, standard
libraries.
Task 4. Geometry figures drawing.
Version 1.0
Developer: Morozova E.S.
Date: 12.04.2026
"""

from task4.parallelogram import Parallelogram
from utils.constants import ALLOWED_COLORS
from utils.input import input_float, check_color, input_str
from utils.output import task4_info

def run():
    """Execute task 4. """
    try:
        task4_info(ALLOWED_COLORS)
        caption = input_str("Enter figure caption: ")
        diagonal1 = input_float(0, 10000, "Enter diagonal 1: ")
        diagonal2 = input_float(0, 10000, "Enter diagonal 2: ")
        angle = input_float(0, 180, "Enter angle(0° < x < 180°): ")
        color = check_color()
        parallelogram = Parallelogram(diagonal1, diagonal2, angle, color)
        print(Parallelogram.get_name())
        print(parallelogram.get_info())

        parallelogram.draw(caption)
    except FileNotFoundError as e:
        print(f"File not found error: {e}")
    except PermissionError as e:
        print(f"Permission error: {e}")

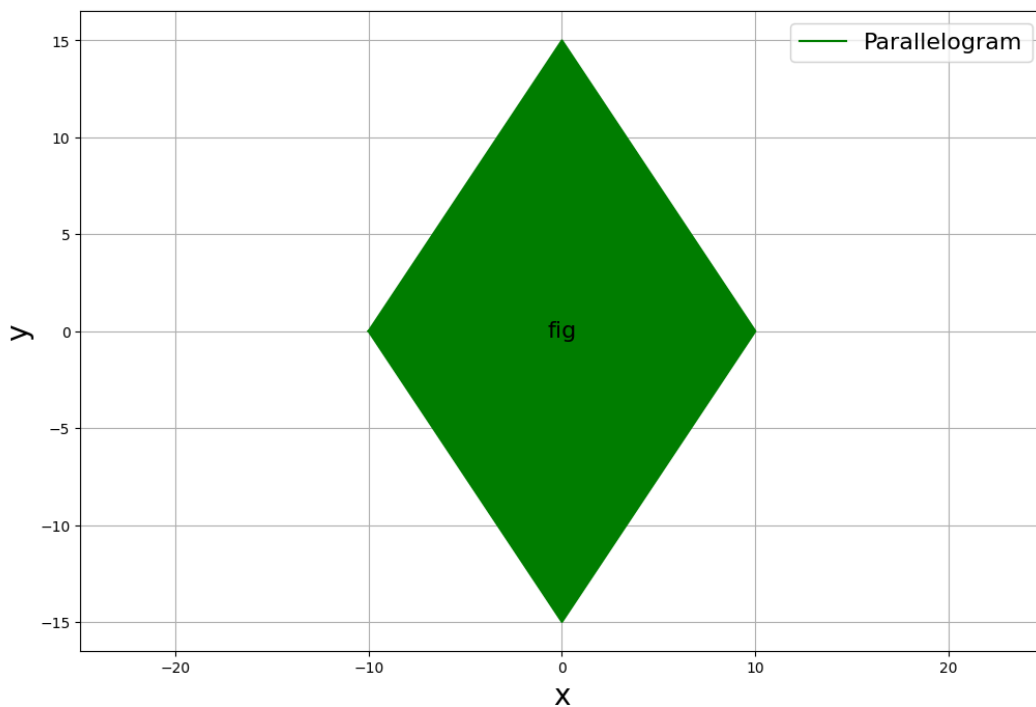
```

```
except ValueError as e:
    print(f"Value error empty array: {e}")
```

```
Task 4 info:
Available colors:
red, orange, yellow, green, blue, violet, purple, pink, brown, black

Enter figure caption: fig
Enter diagonal 1: 20
Enter diagonal 2: 30
Enter angle(0° < x < 180°): 90
Enter color: green

Figure name: Parallelogram
Diagonal 1: 20.0
Diagonal 2: 30.0
Angle: 90.0°
Area: 300.0000
Color: green
█
```



5. В соответствии с заданием своего варианта исследовать возможности библиотеки NumPy при работе с массивами и математическими и статическими операциями. Сформировать целочисленную матрицу $A[n,m]$ с помощью генератора случайных чисел (random).
- а) Библиотека NumPy.
1. Создание массива. Функции `array()` и `values()`.

2. Функции создания массива заданного вида.
3. Индексирование массивов NumPy. Индекс и срез.
4. Операции с массивами. Универсальные (поэлементные) функции.
- б) Математические и статистические операции.

1. Функция `mean()`
2. Функция `median()`
3. Функция `corrcoef()`
4. Дисперсия `var()`.
5. Стандартное отклонение `std()`

Найти сумму модулей отрицательных нечетных элементов.

Вычислить стандартное отклонение для этих значений. Ответ округлите до сотых. Вычисление стандартного отклонения выполнить двумя способами:

через стандартную функцию и через программирование формулы.

```
"""
Task 5. Module for Array Creator.
"""
import numpy as np

class ArrayCreatorMixin:
    """Mixin providing array creation functionality."""
    def create_matrix(self, n: int, m: int):
        """
        Create a random integer matrix with values in range [-100, 100).

        Args:
            n: Number of rows
            m: Number of columns

        Returns:
            np.ndarray: Random integer matrix of shape (n, m)
        """
        return np.random.randint(-100, 100, (n, m))

    def matrix_creating_examples(self):
        """Demonstrate various NumPy array creation functions."""
        print(f"Zeros matrix:\n{np.zeros((2, 2))}")
        print(f"Ones matrix:\n{np.ones((2, 2))}")
        print(f"Diagonal matrix:\n{np.identity(2)}")
        print(f"Eye matrix:\n{np.eye(3, 3, k=1)}")

"""
Task 5. Module for Array Demonstration Mixin.
"""
import numpy as np
```

```

class ArrayDemoMixin:
    """Mixin providing NumPy array demonstration functionality."""
    def matrix_indexing_slicing_example(self, matrix):
        """
        Demonstrate array indexing and slicing.

        Args:
            matrix: 2D NumPy array to demonstrate on
        """
        print(f"Matrix first element: {matrix[0][0]}")
        print(f"Matrix [1..n-1][1..m-1] elements (inside square):\n{matrix[1:len(matrix) - 1, 1: len(matrix[0]) - 1]}")
        print(f"Index of max element: {matrix.argmax()}")
        print(f"Index of min element: {matrix.argmin()}")

    def matrix_operations_example(self):
        """
        Demonstrate universal (element-wise) operations on arrays.
        Shows addition, multiplication, subtraction, division, modulo,
        power and square root.
        """
        a = np.array([[1, 2, 3], [4, 5, 6]], float)
        b = np.array([[10, 11, 12], [13, 14, 15]], float)
        print(f"\nMatrix a:\n{a}")
        print(f"\nMatrix b:\n{b}")
        print(f"\n1) a + b\n{a + b}")
        print(f"\n2) a * b\n{a * b}")
        print(f"\n3) a - b\n{a - b}")
        print(f"\n4) a / b\n{a / b}")
        print(f"\n5) b % a\n{b % a}")
        print(f"\n6) a ** b\n{a ** b}")
        print(f"\n7) sqrt(a)\n{np.sqrt(a)}")

    def statistics_info(self, matrix):
        """
        Calculate and return basic statistics for the matrix.

        Args:
            matrix: 2D NumPy array

        Returns:
            str: Formatted string with mean, median, correlation,
                variance and standard deviation
        """
        return (f"\n--- Statistics info: ---\n"
                f" Mean: {np.mean(matrix)}\n"
                f" Median: {np.median(matrix)}\n"
                f" Corrective coefficients: \n{np.corrcoef(matrix)}\n"
                f" Variance: {np.var(matrix)}\n"
                f" Standard deviation: {np.std(matrix)}\n")

```

Task 5. Module for Array Service.

```

"""
import numpy as np
from task5.array_creator_mixin import ArrayCreatorMixin
from task5.array_demo_mixin import ArrayDemoMixin

class ArrayService(ArrayCreatorMixin, ArrayDemoMixin):
    """
    Main service for NumPy operations.
    Inherits array creation and demonstration methods from mixins.
    """
    def calculate_sum_of_abs_negative_numbers(self, matrix):
        """
        Calculate sum of absolute values of negative odd elements.

        Args:
            matrix: Input NumPy array

        Returns:
            float: Sum of absolute values
        """
        elements = self._select_odd_negative_nums(matrix)
        return np.sum(np.abs(elements))

    def calculate_std_with_built_in_function(self, matrix):
        """
        Calculate standard deviation using built-in np.std().

        Args:
            matrix: Input NumPy array

        Returns:
            float: Standard deviation rounded to 2 decimal places
        """
        elements = self._select_odd_negative_nums(matrix)
        return np.std(elements).round(2)

    def calculate_std_manually(self, matrix):
        """
        Calculate standard deviation manually using formula.

        Args:
            matrix: Input NumPy array

        Returns:
            float: Standard deviation rounded to 2 decimal places
        """
        elements = self._select_odd_negative_nums(matrix)
        amount_of_elements = len(elements)
        mean = np.sum(elements) / amount_of_elements
        deviation_squares = (elements - mean) ** 2
        return (np.sqrt(np.sum(deviation_squares) /
amount_of_elements)).round(2)

```

```

def _select_odd_negative_nums(self, matrix):
    """
    Select elements that are both odd and negative.

    Args:
        matrix: Input NumPy array

    Returns:
        np.ndarray: 1D array of negative odd numbers
    """
    elements = matrix[matrix % 2 == 1]
    return elements[elements < 0]

"""
Lab work 4. File work, classes, serializers, regular expressions, standard
libraries.
Task 5. Matrix operations with NumPy.
Version 1.0
Developer: Morozova E.S.
Date: 12.04.2026
"""

from task5.array_service import ArrayService
from utils.input import input_int
from utils.output import task5_info

def run():
    """Execute task 5. """
    try:
        task5_info()
        n = input_int(1, 100, "Enter n rows: ")
        m = input_int(1, 100, "Enter m columns: ")
        array_service = ArrayService()
        a = array_service.create_matrix(n, m)
        print(f"Created matrix: \n{a}")

        print("\n--- Matrix creating examples: ---")
        array_service.matrix_creating_examples()
        print("\n--- Matrix indexing and slicing examples: ---")
        array_service.matrix_indexing_slicing_example(a)
        print("\n--- Matrix operations examples: ---")
        array_service.matrix_operations_example()

        print(array_service.statistics_info(a))

        print(f"\nMatrix summary of modules of negative numbers:
{array_service.calculate_sum_of_abs_negative_numbers(a)}")
        print(f"Built-in calculated square deviation:
{array_service.calculate_std_with_built_in_function(a)}")
        print(f"Manually calculated square deviation:
{array_service.calculate_std_manually(a)}")
    except ZeroDivisionError as e:
        print(f"ZeroDivisionError: {e}")

```

```
except ValueError as e:
    print(f"ValueError: {e}")
```

```
Task 5 info:
Enter n rows: 5
Enter m columns: 5
Created matrix:
[[ 75  86  64  68  26]
 [ 76  22 -12   1  46]
 [  3  70   3 -90 -90]
 [-46  18  44  -1  -7]
 [ 23 -87  60 -36  34]]

--- Matrix creating examples: ---
Zeros matrix:
[[0. 0.]
 [0. 0.]]
Ones matrix:
[[1. 1.]
 [1. 1.]]
Diagonal matrix:
[[1. 0.]
 [0. 1.]]
Eye matrix:
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 0.]]

--- Matrix indexing and slicing examples: ---
Matrix first element: 75
Matrix [1..n-1][1..m-1] elements (inside square):
[[ 22 -12   1]
 [ 70   3 -90]
 [ 18  44  -1]]
Index of max element: 1
Index of min element: 13
```

```
--- Matrix operations examples: ---

Matrix a:
[[1. 2. 3.]
 [4. 5. 6.]]

Matrix b:
[[10. 11. 12.]
 [13. 14. 15.]]

1) a + b
[[11. 13. 15.]
 [17. 19. 21.]]

2) a * b
[[10. 22. 36.]
 [52. 70. 90.]]

3) a - b
[[-9. -9. -9.]
 [-9. -9. -9.]]

4) a / b
[[0.1      0.18181818 0.25      ]
 [0.30769231 0.35714286 0.4      ]]

5) b % a
[[0. 1. 0.]
 [1. 4. 3.]]

6) a ** b
[[1.00000000e+00 2.04800000e+03 5.31441000e+05]
 [6.71088640e+07 6.10351562e+09 4.70184985e+11]]
```

```
7) sqrt(a)
[[1.      1.41421356 1.73205081]
 [2.      2.23606798 2.44948974]]

--- Statistics info: ---
Mean: 14.0
Median: 22.0
Corrective coefficients:
[[ 1.      -0.12388956  0.73724109  0.05088332 -0.57332747]
 [-0.12388956  1.      0.02761651 -0.88788945  0.09548515]
 [ 0.73724109  0.02761651  1.      0.234023   -0.35337524]
 [ 0.05088332 -0.88788945  0.234023   1.      -0.02232822]
 [-0.57332747  0.09548515 -0.35337524 -0.02232822  1.      ]]
Variance: 2629.28
Standard deviation: 51.2765053411404

Matrix summary of modules of negative numbers: 95
Built-in calculated square deviation: 39.2
Manually calculated square deviation: 39.2
```

6. Используя указанный в задании набор данных (datasets) на ресурсе <https://www.kaggle.com/datasets> исследовать основные возможности библиотеки Pandas. Если какая-то из ссылок не работает, то искать на ресурсе с помощью поиска или в Ylgooogle (пример запроса: spotify dataset kaggle).

См. Анализ Данных Python методичка.pdf

ЗАДАНИЕ А. Библиотека Pandas. Структуры Series и DataFrame

1. Библиотека Pandas. Импорт библиотеки.
2. Структура Series.
3. Создание Series .
4. Функция display.
5. Доступ к элементам Series с использованием .loc или .iloc.
6. Объект DataFrame. Создание.

21	Прогноз погоды	Weather Dataset	Создайте DataFrame из первых 7 дней данных, содержащий только температуру и влажность. Измените индексы на названия дней недели.	Фильтрация данных, пользовательские индексы
----	----------------	---------------------------------	--	---

ЗАДАНИЕ Б. Основные операции и статистический анализ

Получение информации о датафрейме (по каждому параметру)

Индексация и извлечение данных: статистические методы (во сколько раз среднее значение выбранного показателя с максимальным значением другого параметра (например, самый молодой, самый быстрый и т.д.) выше среднего выбранного показателя с минимальным значением)

21	Прогноз погоды	Weather Dataset	Определите, во сколько раз средняя температура (Temperature_c) в самые жаркие дни (верхний дециль) больше, чем в самые холодные дни (нижний дециль) . Ответ округлите до сотых.
----	----------------	---------------------------------	--

```

"""
Task 6. Module for Library service.
"""
import pandas as pd
from IPython.core.display_functions import display

class LibService:
    """Service for weather dataset analysis."""
    def __init__(self):
        """Initialize and load weather dataset."""
        self.__data = self._read_file()

    def _read_file(self):
        """
        Read weather dataset from CSV file.

        Returns:
            pd.DataFrame: Loaded weather data
        """
        data = pd.read_csv("task6/weatherHistory.csv")
        return data

```

```

def select_first_records(self, num_of_records: int):
    """
    Select first N records at 12:00 PM showing temperature and humidity.

    Args:
        num_of_records: Number of records to display
    """
    data = self.__data.copy()
    data["Formatted Date"] = pd.to_datetime(data["Formatted Date"],
utc=True)
    data.index = data["Formatted Date"].dt.day_name()
    mask = data["Formatted Date"].dt.hour == 12
    filtered_data = data[mask].head(num_of_records)
    display(filtered_data[["Temperature (C)", "Humidity"]])

def statistical_analysis(self):
    """
    Perform decile analysis: compare average temperature of hottest days
    (above 90th percentile) vs coldest days (below 10th percentile).

    Prints ratio of hot mean to cold mean.
    """
    temperatures = self.__data["Temperature (C)"]
    quant_10 = temperatures.quantile(0.1)
    quant_90 = temperatures.quantile(0.9)

    cold_temps = temperatures[temperatures <= quant_10]
    cold_mean = cold_temps.mean()
    hot_temps = temperatures[temperatures >= quant_90]
    hot_mean = hot_temps.mean()
    display(f"Average temperature on hottest days is higher than on
coldest days (quantile_90 / quantile_10) in: {round(hot_mean /
abs(cold_mean), 2)}")

"""
Lab work 4. File work, classes, serializers, regular expressions, standard
libraries.
Task 6. CSV-file statistical analysis.
Version 1.0
Developer: Morozova E.S.
Date: 12.04.2026
"""

from task6.lib_service import LibService
from utils.output import task6_info

def run():
    """Execute task 6: weather dataset analysis."""
    try:
        task6_info()
        lib_service = LibService()
        print("Task A. Pandas Library. Structures Series and DataFrame\n")
        lib_service.select_first_records(7)

```

```
print("\nTask B. Basic operations and statistical analysis\n")
lib_service.statistical_analysis()
except FileNotFoundError as e:
    print()
```

Task 6 info:

Task A. Pandas Library. Structures Series and DataFrame

Formatted Date	Temperature (C)	Humidity
Saturday	18.877778	0.47
Monday	21.050000	0.40
Tuesday	17.361111	0.71
Wednesday	9.888889	0.71
Thursday	9.077778	0.65
Friday	15.150000	0.50
Saturday	17.144444	0.43

Task B. Basic operations and statistical analysis

Average temperature on hottest days is higher than on coldest days (quantile_90 / quantile_10) in: 7.9